P08 Enemy Patrol

CRE132 worksheet

What you'll make

Cats that walk back and forth along the platforms. They turn round when they reach a wall, and before they walk off the edge of a platform. Jump on a cat's head and it turns into a ghost and floats away while you bounce up. Walk into its side and you lose 1 health.

You'll learn how two short **raycasts** give an enemy enough "sight" to patrol any platform, and how comparing two positions tells you whether the player came from above or from the side.

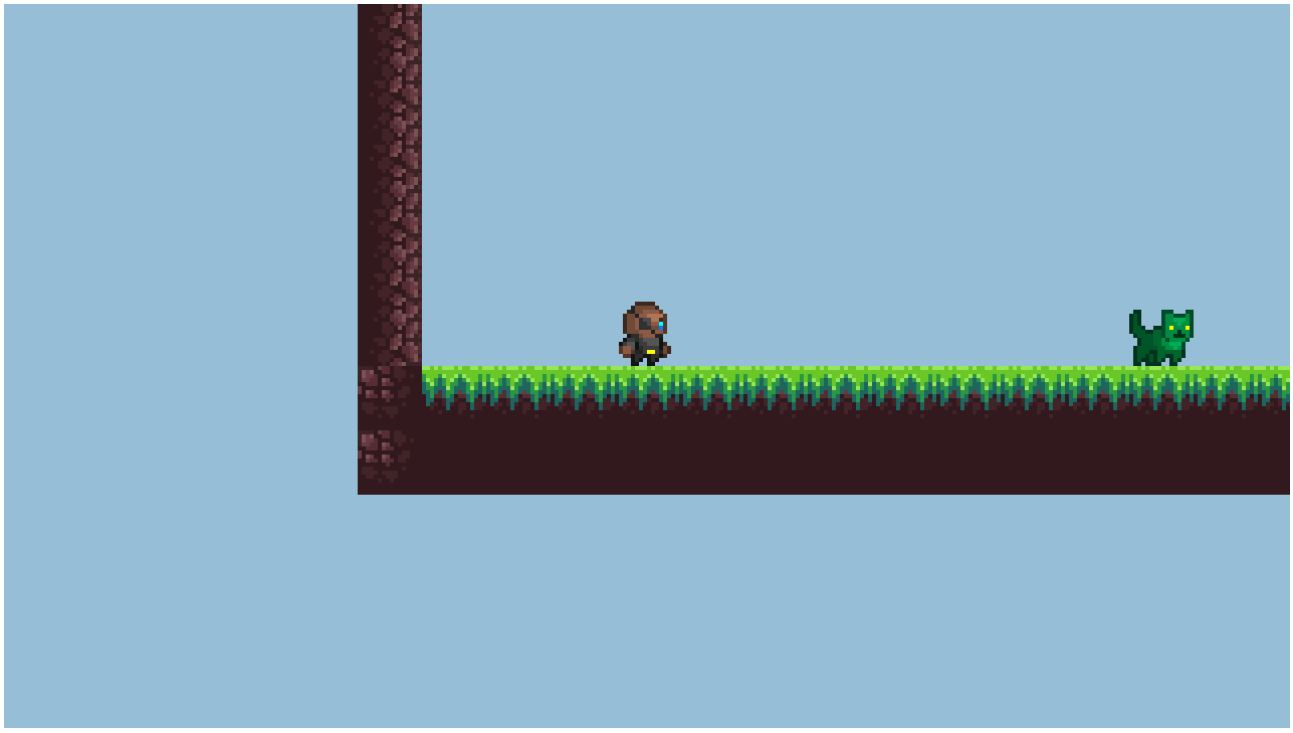


Figure 1: The start of the level: the player by the left wall, with a cat patrolling further along.

Before you start

1. Unzip `CRE132_P08_EnemyPatrol_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_P08_EnemyPatrol_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Platformer/P08_EnemyPatrol`.
3. Press **Play**. You can run (A/D or arrows) and jump (Space), and the health bar shows your 3 health. The five **Cat** objects play their walk animation but stay where they are. Press Play again to stop.

Each **Cat** (inside **Enemies**) already has a Sprite Renderer, an Animator, a Box Collider 2D and a **Rigidbody 2D** with **Body Type** set to **Kinematic**. A kinematic body only moves when a script sets its velocity, and the player can't shove it around.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Walking and turning: EnemyPatrol

Make a folder `Assets/_game/Scripts/P08_EnemyPatrol` and create a script in it called `EnemyPatrol`:

```
// Walks back and forth along a platform, turning around at walls and before walking
// off an edge.
// Uses a Kinematic Rigidbody2D, so it moves only by its velocity and can't be pushed
// by the player.
using UnityEngine;

public class EnemyPatrol : MonoBehaviour
{
    [SerializeField] private float speed = 2f;
    [SerializeField] private LayerMask groundLayers;

    // How far ahead of the middle the wall and edge checks look.
    [SerializeField] private float lookAhead = 0.5f;

    private Rigidbody2D body;
    private SpriteRenderer spriteRenderer;

    // 1 = walking right, -1 = walking left.
    private float direction = 1f;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        spriteRenderer = GetComponent<SpriteRenderer>();
    }

    private void FixedUpdate()
    {
        Vector2 position = transform.position;

        // Wall check: a short ray forwards at waist height.
        Vector2 wallCheckStart = position + new Vector2(direction * lookAhead, 0.4f);
        RaycastHit2D wall = Physics2D.Raycast(wallCheckStart, new Vector2(direction, 0f),
            0.1f, groundLayers);

        // Edge check: a ray straight down just in front of the feet. Nothing below
        // means the platform ends.
        Vector2 edgeCheckStart = position + new Vector2(direction * lookAhead, 0.1f);
        RaycastHit2D groundAhead = Physics2D.Raycast(edgeCheckStart, Vector2.down, 0.5f,
            groundLayers);

        if (wall.collider != null || groundAhead.collider == null)
        {
            TurnAround();
        }

        body.linearVelocity = new Vector2(direction * speed, 0f);
    }

    private void TurnAround()
    {
        direction = -direction;

        // The art faces right, so flip it when walking left.
        spriteRenderer.flipX = direction < 0f;
    }
```

```
    private void OnDrawGizmosSelected()
    {
        Vector3 position = transform.position;
        Gizmos.color = Color.yellow;
        Gizmos.DrawLine(position + new Vector3(direction * lookAhead, 0.4f, 0f),
            position + new Vector3(direction * (lookAhead + 0.1f), 0.4f, 0f));
        Gizmos.DrawLine(position + new Vector3(direction * lookAhead, 0.1f, 0f),
            position + new Vector3(direction * lookAhead, -0.4f, 0f));
    }
}
```

What's going on:

- **`direction`** is 1 (right) or −1 (left). Every physics step, `FixedUpdate` sets the velocity to `direction * speed` sideways, so turning round is just `direction = -direction`.
- **`Physics2D.Raycast`** shoots an invisible line and tells you the first collider it hits on the `groundLayers`. The **wall check** looks a short way forwards at waist height. The **edge check** looks straight down from just in front of the feet: if it hits nothing, the platform ends there.
- **`wall.collider != null || groundAhead.collider == null`** reads "there's a wall, or there's no ground ahead". Either one means turn round. `||` means "or".
- **`OnDrawGizmosSelected`** draws both rays as yellow lines in the Scene view while a cat is selected, so you can see what it's checking.

Add it: select all five **Cat** objects (in **Enemies**, click the first and Shift-click the last) and click **Add Component > Enemy Patrol**. Leave **Speed** at 2 and **Look Ahead** at 0.5. Set **Ground Layers** to **Ground**: the tilemap is on that layer.

Test it: press Play. The cats walk, turn at the low wall and turn back before each platform edge. Select a cat in the Hierarchy while it's playing and look at the yellow rays in the Scene view.

2. Jumping on heads: Stompable

Create `Stompable` in the same `P08_EnemyPatrol` folder:

```
// Jump on this enemy's head to defeat it (and bounce off). Walking into its side
// hurts the player instead.
using UnityEngine;

public class Stompable : MonoBehaviour
{
    [SerializeField] private float bounceSpeed = 12f;
    [SerializeField] private int damageToPlayer = 1;

    // The player's feet must be at least this far above the enemy's feet to count as
    // a stomp.
    [SerializeField] private float stompHeight = 0.5f;

    // Played where the enemy was (a ghost floating away), then removed after
    // deathEffectTime.
    [SerializeField] private GameObject deathEffect;
    [SerializeField] private float deathEffectTime = 0.6f;

    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (!collision.gameObject.CompareTag("Player"))
        {
            return;
```

```
        }

        bool landedOnTop = collision.transform.position.y >
            transform.position.y + stompHeight;
        if (landedOnTop)
        {
            Rigidbody2D playerBody = collision.rigidbody;
            playerBody.linearVelocity = new Vector2(playerBody.linearVelocity.x,
                bounceSpeed);
            Defeat();
        }
        else
        {
            Health playerHealth = collision.gameObject.GetComponent<Health>();
            if (playerHealth != null)
            {
                playerHealth.TakeDamage(damageToPlayer);
            }
        }
    }

    private void Defeat()
    {
        if (deathEffect != null)
        {
            GameObject effect = Instantiate(deathEffect, transform.position,
                Quaternion.identity);
            effect.GetComponent<SpriteRenderer>().flipX =
                GetComponent<SpriteRenderer>().flipX;
            Destroy(effect, deathEffectTime);
        }
        Destroy(gameObject);
    }
}
```

- **`OnCollisionEnter2D`** runs once when the player bumps into the cat. The cat's collider is solid (not a trigger), so this is a collision, not a trigger.
- **`CompareTag("Player")`** ignores everything except the player. The Player object already has the **Player** tag.
- **`landedOnTop`** compares heights. Both objects have their pivot at their feet, so if the player's feet are more than `stompHeight` above the cat's feet, the player came from above. Otherwise it was a side hit.
- **A stomp** sets the player's upward speed to `bounceSpeed` (keeping the sideways speed), then calls `Defeat`. `Defeat` spawns the ghost facing the same way as the cat, removes it after `deathEffectTime` seconds with `Destroy(effect, deathEffectTime)`, and destroys the cat.

Add it: with all five cats selected, **Add Component > Stompable**. Leave **Bounce Speed** 12, **Damage To Player** 1, **Stomp Height** 0.5 and **Death Effect Time** 0.6. Drag the **Cat Ghost** prefab from `Assets/_game/Prefabs/Characters` into the **Death Effect** slot.

Test it: jump on a cat. It becomes a ghost and you bounce up. Walk into a cat's side and the health bar drops by 1. Lose all 3 and the level restarts.

Check your work

- ☐ Cats walk back and forth, turning at the low wall and at platform edges. None walk off.
- ☐ Selecting a cat shows its two yellow check rays.
- ☐ Landing on a cat turns it into a floating ghost and bounces you up.

- ☐ Walking into a cat's side costs 1 health. At 0 health the level restarts.
- ☐ Compare with `CRE132_P08_EnemyPatrol_Finished` if something behaves differently.

Key points

- **Two short raycasts** (one forwards, one down) are enough for an enemy to patrol any platform without falling off.
- **A kinematic body** moves only by the velocity you give it, so the player can't push enemies around.
- **Comparing positions** is a simple, readable way to tell "from above" from "from the side".
- Notice that a side hit only hurts once per touch, and you can stay touching the cat. P09 adds knockback and invincibility to fix that.

If something goes wrong

- **The cats turn round on the spot, over and over: Ground Layers** is empty (**Nothing**), so the edge check never finds ground. Set it to **Ground**.
- **The cats hang half over the edge before turning: Look Ahead** is 0, so the edge ray starts under the middle of the cat. Set it back to 0.5.
- **The player can push the cats around:** the Rigidbody 2D's **Body Type** has been changed from **Kinematic**.
- **Jumping on a cat hurts you instead:** check **Stomp Height** is 0.5. A much bigger value means you can never be high enough.
- **A stomped cat just vanishes, with no ghost:** the **Death Effect** slot is empty.
- **Nothing happens when you touch a cat:** the Player must keep its **Player** tag, and Stompable must be on the **Cat** itself.

Try it

1. Give each cat a different **Speed** in the Inspector.
2. Make a cat that also turns round every few seconds, even on open ground. (Hint: a timer like S01's `nextHitTime`.)
3. Make the cats chase the player when the player is close. (T06 does this for top-down games.)